

WEEK 12 PROJECT

MongoDB CRUD Operations

Student Database practice and Watch Store use case

Detail	Information
Prepared by	Tamil
Tool used	MongoDB Shell (mongosh)
Database 1	studentDB (collection: students)
Database 2	watchStore (collections: men, women, smart)
Operations covered	Create, Insert, Read, Update, Delete, Query Operators, Array Updates

Objective

This project documents hands-on practice with the core MongoDB operations (CRUD). Part A works through seven guided tasks on a students collection. Part B applies the same skills to a small e-commerce use case, a Watch Store, with separate collections for men, women and smart watches.

Tasks at a glance

No.	Task	Key commands
1	Database and collection setup	use, createCollection
2	Insert operations	insertOne, insertMany
3	Read operations	find, find with filter
4	Update operations	updateOne, updateMany
5	Delete operations	deleteOne, deleteMany
6	Query operators	\$gt, \$lt, \$in, \$and, \$or, \$exists, \$push
7	Use case: Watch Store	insert, update and search on three collections

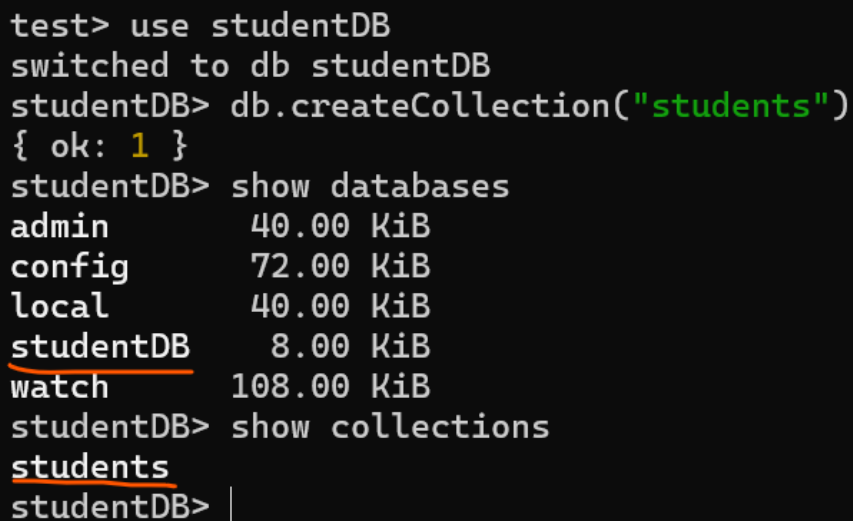
Student Database

Task 1: Database & Collection Setup

Goal: create a new database called studentDB and a collection called students.

COMMANDS

```
use studentDB
db.createCollection("students")
show databases
show collections
```



```
test> use studentDB
switched to db studentDB
studentDB> db.createCollection("students")
{ ok: 1 }
studentDB> show databases
admin          40.00 KiB
config         72.00 KiB
local          40.00 KiB
studentDB      8.00 KiB
watch         108.00 KiB
studentDB> show collections
students
studentDB> |
```

Figure 1: Creating studentDB and the students collection

Result: MongoDB switched to studentDB, returned { ok: 1 } for the new collection, and both studentDB and students now appear in the listings.

Tip: MongoDB only lists a database once it contains data or a collection, so an empty database will not show up in "show databases".

Task 2: Insert Operations

2.1 Insert one document

COMMAND

```
db.students.insertOne({ name: "Tamil", age: 30, course: "MERN", batch: 30, status: "Active" })
```

```
studentDB> db.students.insertOne({name : "Tamil", age: 30, course: 'MERN', batch : 30 ,status:'Active'})
{
  acknowledged: true,
  insertedId: ObjectId('6ac2776064d65068db9ae84e')
}
studentDB> db.students.find()
[
  { _id: ObjectId('6ac276f564d65068db9ae84d') },
  {
    _id: ObjectId('6ac2776064d65068db9ae84e'),
    name: 'Tamil',
    age: 30,
    course: 'MERN',
    batch: 30,
    status: 'Active'
  }
]
studentDB> |
```

Figure 2: insertOne adds a single student and find() confirms it

Result: acknowledged: true with a new insertedId means the document was saved.

2.2 Insert multiple documents

COMMAND

```
db.students.insertMany([
  { name: "Gracy", age: 23, course: "Python", batch: 20, status: "Active" },
  { name: "Govind", age: 27, course: "Java", batch: 25, status: "Active" }
])
```

```
studentDB> db.students.insertMany([ {name: 'Gracy', age: 23, course: 'Python', batch: 20, status: 'Active'}, {name: 'Govind', age: 27, course: 'Java', batch: 25, status: 'Active'} ])
{
  acknowledged: true,
  insertedIds: [
    '0': ObjectId('6ac2776064d65068db9ae84f'),
    '1': ObjectId('6ac2776064d65068db9ae84e')
  ]
}
```

Figure 3: insertMany adds two students in one command

Result: Two ObjectIds are returned in insertedIds, one for each new document.

Task 3: Read Operations

3.1 Fetch all documents

COMMAND

```
db.students.find()
```

```
studentDB> db.students.find()
[
  {
    _id: ObjectId('6ac276f564d65068db9ae84d'),
    name: 'Tamil',
    age: 30,
    course: 'MERN',
    batch: 30,
    status: 'Active'
  },
  {
    _id: ObjectId('6ac2789e64d65068db9ae84f'),
    name: 'Gracy',
    age: 23,
    course: 'Python',
    batch: 20,
    status: 'Active'
  },
  {
    _id: ObjectId('6ac2789e64d65068db9ae850'),
    name: 'Govind',
    age: 27,
    course: 'Java',
    batch: 25,
    status: 'Active'
  }
]
studentDB> |
```

Figure 4: `find()` returns every document in the collection

3.2 Fetch with a filter

A filter returns only the documents that match a condition. Here, only students enrolled in the MERN course.

COMMAND

```
db.students.find({ course: 'MERN' })
```

```
studentDB> db.students.find({course:'MERN'})
[
  {
    _id: ObjectId('6ac2776064d65068db9ae84e'),
    name: 'Tamil',
    age: 30,
    course: 'MERN',
    batch: 30,
    status: 'Active'
  }
]
studentDB> |
```

Figure 5: Filtering students by course

Result: Only Tamil (course: MERN) is returned; Gracy and Govind are skipped.

Task 4: Update Operations

4.1 Update a single document

updateOne changes the first document that matches the filter. \$set replaces the value of a field.

COMMAND

```
db.students.updateOne({ name: 'Gracy' }, { $set: { status: 'Inactive' } })
```

```
studentDB> db.students.updateOne({name:'Gracy'},{$set : {status:'Inactive'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB> db.students.find()
[
  { _id: ObjectId('6ac276f864d65068db9ae84d') },
  {
    _id: ObjectId('6ac2776064d65068db9ae84e'),
    name: 'Tamil',
    age: 30,
    course: 'MERN',
    batch: 30,
    status: 'Active'
  },
  {
    _id: ObjectId('6ac2789e64d65068db9ae84f'),
    name: 'Gracy',
    age: 23,
    course: 'Python',
    batch: 20,
    status: 'Inactive'
  },
  {
    _id: ObjectId('6ac2789e64d65068db9ae850'),
    name: 'Govind',
    age: 27,
    course: 'Java',
    batch: 25,
    status: 'Active'
  }
]
```

Figure 6: Gracy's status changed from Active to Inactive

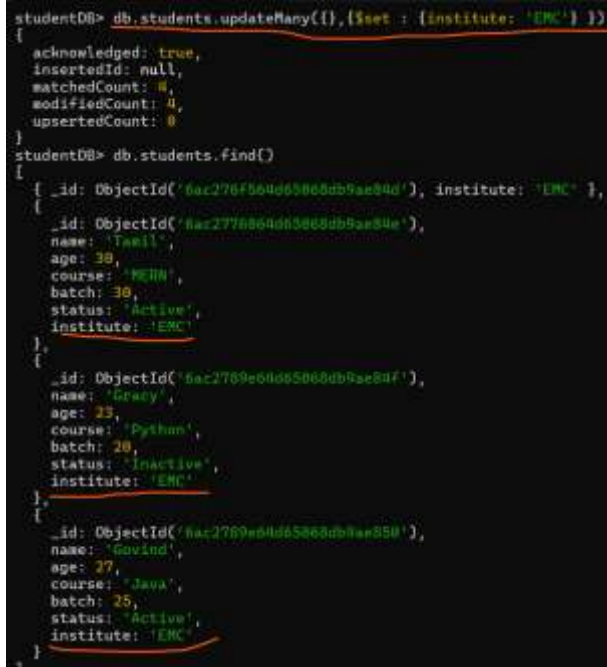
Result: matchedCount: 1 and modifiedCount: 1 confirm exactly one document was changed.

4.2 Update multiple documents

updateMany changes every matching document. An empty filter {} matches all documents, so the new field is added everywhere.

COMMAND

```
db.students.updateMany({}, { $set: { institute: 'EMC' } })
```



```
studentDB> db.students.updateMany({}, {$set: {institute: 'EMC'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 4,
  modifiedCount: 4,
  upsertedCount: 0
}
studentDB> db.students.find()
[
  {
    _id: ObjectId('6ac276f564d65868db9ae84d'), institute: 'EMC',
    {
      _id: ObjectId('6ac2776864d65868db9ae84e'),
      name: 'Taeil',
      age: 38,
      course: 'WEBM',
      batch: 38,
      status: 'Active',
      institute: 'EMC'
    }
  },
  {
    _id: ObjectId('6ac2769e64d65868db9ae84f'),
    name: 'Gracy',
    age: 23,
    course: 'Python',
    batch: 28,
    status: 'Inactive',
    institute: 'EMC'
  },
  {
    _id: ObjectId('6ac2769e64d65868db9ae850'),
    name: 'Govind',
    age: 27,
    course: 'Java',
    batch: 25,
    status: 'Active',
    institute: 'EMC'
  }
]
```

Figure 7: updateMany adds the institute field to all documents

Result: matchedCount: 4 and modifiedCount: 4, so all four documents now include institute: "EMC".

Task 5: Delete Operations

5.1 Delete one document based on a condition

COMMAND

```
db.students.deleteOne({ status: 'Inactive' })
```

```
]
studentDB> db.students.deleteOne({status:'Inactive'})
{ acknowledged: true, deletedCount: 1 }
studentDB> db.students.find()
[
  { _id: ObjectId('6ac276f564d65068db9ae84d'), institute: 'EMC' },
  {
    _id: ObjectId('6ac2776064d65068db9ae84e'),
    name: 'Tamil',
    age: 30,
    course: 'MERN',
    batch: 30,
    status: 'Active',
    institute: 'EMC'
  },
  {
    _id: ObjectId('6ac2789e64d65068db9ae850'),
    name: 'Govind',
    age: 27,
    course: 'Java',
    batch: 25,
    status: 'Active',
    institute: 'EMC'
  }
]
studentDB> |
```

Figure 8: deleteOne removes the first Inactive student

Result: deletedCount: 1. Gracy (Inactive) is gone and the other documents are untouched.

5.2 Delete multiple documents

COMMAND

```
db.students.deleteMany({ course: 'Java' })
```

```
]
studentDB> db.students.deleteMany({course:'Java'})
{ acknowledged: true, deletedCount: 1 }
studentDB> db.students.find()
[
  { _id: ObjectId('6ac276f564d65068db9ae84d'), institute: 'EMC' },
  {
    _id: ObjectId('6ac2776064d65068db9ae84e'),
    name: 'Tamil',
    age: 30,
    course: 'MERN',
    batch: 30,
    status: 'Active',
    institute: 'EMC'
  }
]
studentDB> |
```

Figure 9: deleteMany removes every student enrolled in Java

Result: deletedCount: 1. Govind (Java) is removed, leaving only Tamil and the base document.

Practice only: To delete every document in a collection, use an empty filter: `db.students.deleteMany({})`. The collection stays, but it becomes empty. Never run this on real data.

Task 6: Query Operators

Query operators start with a \$ sign and let you build smarter filters. The table below is a quick reference, followed by a worked example of each.

Operator	Meaning	Example
\$gt	Greater than	{ price: { \$gt: 5000 } }
\$lt	Less than	{ price: { \$lt: 5000 } }
\$in	Matches any value in a list	{ strap: { \$in: ['leather'] } }
\$and	All conditions must be true	{ \$and: [{ category: 'men' }, { price: { \$lt: 3000 } }] }
\$or	At least one condition is true	{ \$or: [{ category: 'men' }, { price: { \$lt: 3000 } }] }
\$exists	Field is present in the document	{ discount: { \$exists: true } }
\$push	Adds a value to an array field	{ \$push: { tags: 'Bestseller' } }

Note: These examples run on the men collection of the watchStore database, which is created in Task 7.

6.1 \$gt (greater than)

COMMAND

```
db.men.find({ price: { $gt: 5000 } })
```

```
}
]
watchStore> db.men.find({price:{ $gt : 5000}})
[
  {
    _id: ObjectId('6ac2805b64d65068db9ae854'),
    brand: 'Fossil',
    price: 9995,
    strap: 'leather',
    color: 'black',
    category: 'men',
    discount: 1000
  }
]
```

Figure 10: Watches priced above 5000

Result: Only the Fossil watch (9995) is above 5000.

6.2 \$lt (less than)

COMMAND

```
db.men.find({ price: { $lt: 5000 } })
```

```
watchStore> db.men.find({price : { $lt:5000}})
[
  {
    _id: ObjectId('6ac2805b64d65068db9ae851'),
    brand: 'Titan',
    price: 4995,
    strap: 'leather',
    color: 'brown',
    category: 'men'
  },
  {
    _id: ObjectId('6ac2805b64d65068db9ae852'),
    brand: 'Fastrack',
    price: 1995,
    strap: 'silicone',
    color: 'black',
    category: 'men'
  },
  {
    _id: ObjectId('6ac2805b64d65068db9ae853'),
    brand: 'Casio',
    price: 3295,
    strap: 'steel',
    color: 'silver',
    category: 'men'
  },
  {
    _id: ObjectId('6ac2805b64d65068db9ae855'),
    brand: 'Noise',
    price: 2499,
    strap: 'silicone',
    color: 'grey',
    category: 'men'
  }
]
watchStore> |
```

Figure 11: Watches priced below 5000

Result: Titan, Fastrack, Casio and Noise are all below 5000.

6.3 \$in (match any value in a list)

COMMAND

```
db.men.find({ strap: { $in: ['leather'] } })
```



```
watchStore> db.men.find({strap: {$in: ['leather'] } })
[
  {
    _id: ObjectId('6ac2805b64d65068db9ae851'),
    brand: 'Titan',
    price: 4995,
    strap: 'leather',
    color: 'brown',
    category: 'men'
  },
  {
    _id: ObjectId('6ac2805b64d65068db9ae854'),
    brand: 'Fossil',
    price: 9995,
    strap: 'leather',
    color: 'black',
    category: 'men',
    discount: 1000
  }
]
watchStore> |
```

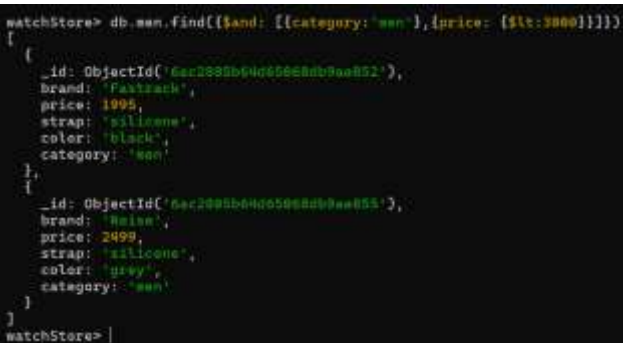
Figure 12: Watches with a leather strap

Result: Titan and Fossil both have a leather strap. Add more values to the list to match several straps at once.

6.4 \$and (all conditions must match)

COMMAND

```
db.men.find({ $and: [{ category: 'men' }, { price: { $lt: 3000 } }] })
```



```
watchStore> db.men.find({$and: [{category: 'men'}, {price: {$lt: 3000}]}})
[
  {
    _id: ObjectId('6ac2805b64d65068db9ae851'),
    brand: 'Fastrack',
    price: 1995,
    strap: 'silicone',
    color: 'black',
    category: 'men'
  },
  {
    _id: ObjectId('6ac2805b64d65068db9ae853'),
    brand: 'Noise',
    price: 2499,
    strap: 'silicone',
    color: 'grey',
    category: 'men'
  }
]
watchStore> |
```

Figure 13: Men's watches priced below 3000

Result: Fastrack (1995) and Noise (2499) pass both conditions.

6.5 \$or (any condition may match)

COMMAND

```
db.men.find({ $or: [{ category: 'men' }, { price: { $lt: 3000 } }] })
```

```
]
watchStore> db.men.find({$or: [{category: 'men'}, {price: {$lt: 3000}}]})
[
  {
    _id: ObjectId('6ac2805b64d65068db9ae851'),
    brand: 'Titan',
    price: 4995,
    strap: 'leather',
    color: 'brown',
    category: 'men'
  },
  {
    _id: ObjectId('6ac2805b64d65068db9ae852'),
    brand: 'Fastrack',
    price: 1995,
    strap: 'silicone',
    color: 'black',
    category: 'men'
  },
  {
    _id: ObjectId('6ac2805b64d65068db9ae853'),
    brand: 'Casio',
    price: 3295,
    strap: 'steel',
    color: 'silver',
    category: 'men'
  },
  {
    _id: ObjectId('6ac2805b64d65068db9ae854'),
    brand: 'Fossil',
    price: 9995,
    strap: 'leather',
    color: 'black',
    category: 'men',
    discount: 1000
  },
  {
    _id: ObjectId('6ac2805b64d65068db9ae855'),
    brand: 'Noise',
    price: 2499,
    strap: 'silicone',
    color: 'grey',
    category: 'men'
  }
]
watchStore> |
```

Figure 14: Watches that are men's OR cheaper than 3000

Result: All five watches are returned because each one satisfies at least one condition.

6.6 \$exists (field is present)

COMMAND

```
db.men.find({ discount: { $exists: true } })
```

```
]
watchStore> db.men.find({discount : {$exists:true}})
[
  {
    _id: ObjectId('6ac2805b64d65068db9ae854'),
    brand: 'Fossil',
    price: 9995,
    strap: 'leather',
    color: 'black',
    category: 'men',
    discount: 1000
  }
]
watchStore> |
```

Figure 15: Watches that have a discount field

Result: Only Fossil has a discount field (1000).

6.7 \$push (add to an array)

\$push adds a value to an array field. If the field does not exist yet, MongoDB creates it.

COMMANDS

```
db.men.updateMany({}, { $push: { tags: 'Bestseller' } })
db.men.updateMany({}, { $push: { tags: 'Trending' } })
db.men.find()
```

```
2 |
watchStore> db.men.updateMany({}, {$push : {tags : 'Bestseller'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 5,
  modifiedCount: 5,
  upsertedCount: 0
}
watchStore> db.men.updateMany({}, {$push : {tags : 'Trending'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 5,
  modifiedCount: 5,
  upsertedCount: 0
}
watchStore> db.men.find()
[
```

Figure 16: Pushing the tags Bestseller and Trending to all watches

```

}
watchStore> db.watches.find()
[
  {
    _id: ObjectId('6ac2885b64d65668db9ae851'),
    brand: 'Titan',
    price: 4995,
    strap: 'leather',
    color: 'brown',
    category: 'men',
    tags: [ 'Bestseller', 'Trending' ]
  },
  {
    _id: ObjectId('6ac2885b64d65668db9ae852'),
    brand: 'Fastrack',
    price: 1995,
    strap: 'silicone',
    color: 'black',
    category: 'men',
    tags: [ 'Bestseller', 'Trending' ]
  },
  {
    _id: ObjectId('6ac2885b64d65668db9ae853'),
    brand: 'Casio',
    price: 3295,
    strap: 'steel',
    color: 'silver',
    category: 'men',
    tags: [ 'Bestseller', 'Trending' ]
  },
]

```

Figure 17: Each watch now has tags: ["Bestseller", "Trending"]

Result: matchedCount: 5 and modifiedCount: 5 for both updates. Every watch now carries an array with two tags.

Tip: To add several values in one command, use \$each: { \$push: { tags: { \$each: ["Flash", "Trending"] } } }. Use \$addToSet instead of \$push to avoid duplicate tags.

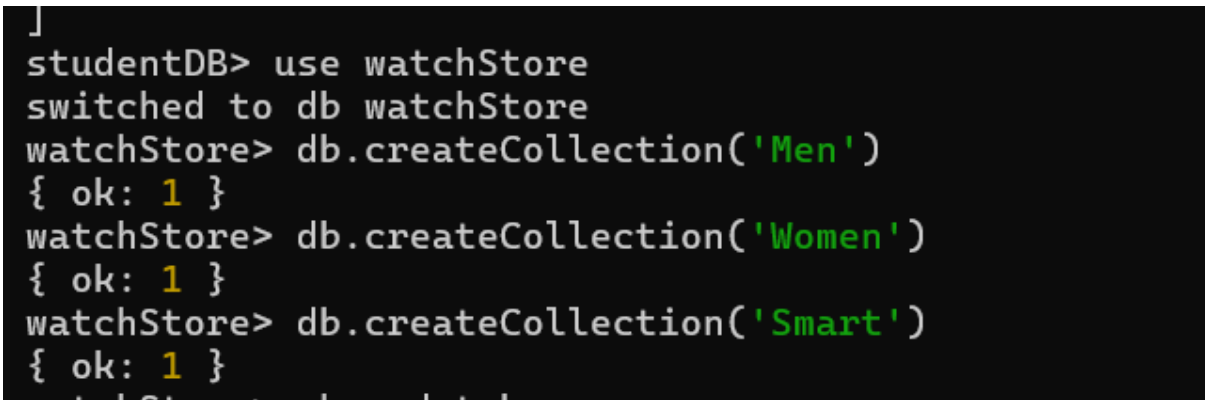
Use Case: Watch Store

To apply the operations in a realistic setting, a small e-commerce product listing was built. Each watch has the fields brand, price, strap, color and category, and the products are grouped into three collections: men, women and smart.

Step 1: Create the database and collections

COMMANDS

```
use watchStore
db.createCollection('Men')
db.createCollection('Women')
db.createCollection('Smart')
```



```
]
studentDB> use watchStore
switched to db watchStore
watchStore> db.createCollection('Men')
{ ok: 1 }
watchStore> db.createCollection('Women')
{ ok: 1 }
watchStore> db.createCollection('Smart')
{ ok: 1 }
```

Figure 18: Creating the watchStore database and its collections

Step 2: Insert data into the collections

2.1 Men watches

COMMAND

```
db.men.insertMany([
  { brand: "Titan", price: 4995, strap: "leather",
    color: "brown", category: "men" },
  { brand: "Fastrack", price: 1995, strap: "silicone",
    color: "black", category: "men" },
  { brand: "Casio", price: 3295, strap: "steel",
    color: "silver", category: "men" },
  { brand: "Fossil", price: 9995, strap: "leather",
    color: "black", category: "men" },
  { brand: "Noise", price: 2499, strap: "silicone",
    color: "grey", category: "men" }
])
```

```
watchStore> db.men.insertMany([
|   { brand: "Titan",   price: 4995,   strap: "leather",   color: "brown",   category: "men" },
|   { brand: "Fastrack", price: 1995,   strap: "silicone", color: "black",   category: "men" },
|   { brand: "Casio",   price: 3295,   strap: "steel",    color: "silver",  category: "men" },
|   { brand: "Fossil",  price: 9995,   strap: "leather",  color: "black",   category: "men" },
|   { brand: "Noise",   price: 2499,   strap: "silicone", color: "grey",    category: "men" }])
|
| {
|   acknowledged: true,
|   insertedIds: {
|     '0': ObjectId('6ac2805b64d65068db9ae851'),
|     '1': ObjectId('6ac2805b64d65068db9ae852'),
|     '2': ObjectId('6ac2805b64d65068db9ae853'),
|     '3': ObjectId('6ac2805b64d65068db9ae854'),
|     '4': ObjectId('6ac2805b64d65068db9ae855')
|   }
| }
watchStore> db.men.find()
[
```

Figure 19: Five men's watches inserted

2.2 Women watches

COMMAND

```
db.women.insertMany([
  { brand: "Titan", price: 4495, strap: "steel",
    color: "rose gold", category: "women" },
  { brand: "Fastrack", price: 1695, strap: "leather",
    color: "pink", category: "women" },
  { brand: "Casio", price: 2995, strap: "steel",
    color: "silver", category: "women" },
  { brand: "Fossil", price: 8995, strap: "steel",
    color: "gold", category: "women" },
  { brand: "Sonata", price: 1195, strap: "leather",
    color: "brown", category: "women" }
])
```

```
watchStore> db.women.insertMany([
|   { brand: "Titan", price: 4495, strap: "steel", color: "rose gold", category: "women" },
|   { brand: "Fastrack", price: 1695, strap: "leather", color: "pink", category: "women" },
|   { brand: "Casio", price: 2995, strap: "steel", color: "silver", category: "women" },
|   { brand: "Fossil", price: 8995, strap: "steel", color: "gold", category: "women" },
|   { brand: "Sonata", price: 1195, strap: "leather", color: "brown", category: "women" }])
|
| {
|   acknowledged: true,
|   insertedIds: {
|     '0': ObjectId('6ac2820864d65868db9ae856'),
|     '1': ObjectId('6ac2820864d65868db9ae857'),
|     '2': ObjectId('6ac2820864d65868db9ae858'),
|     '3': ObjectId('6ac2820864d65868db9ae859'),
|     '4': ObjectId('6ac2820864d65868db9ae85a')
|   }
| }
watchStore> |
```

Figure 20: Five women's watches inserted

2.3 Smart watches

COMMAND

```
db.smart.insertMany([
  { brand: "Noise", price: 2499, strap: "silicone",
    color: "black", category: "smart" },
  { brand: "boAt", price: 1799, strap: "silicone",
    color: "grey", category: "smart" },
  { brand: "Fire-Boltt", price: 1599, strap: "silicone",
    color: "blue", category: "smart" },
  { brand: "Amazfit", price: 7999, strap: "silicone",
    color: "black", category: "smart" },
  { brand: "Samsung", price: 21999, strap: "silicone",
    color: "silver", category: "smart" }
])
```

```
watchStore> db.smart.insertMany([
|   { brand: "Noise", price: 2499, strap: "silicone", color: "black", category: "smart" },
|   { brand: "boAt", price: 1799, strap: "silicone", color: "grey", category: "smart" },
|   { brand: "Fire-Boltt", price: 1599, strap: "silicone", color: "blue", category: "smart" },
|   { brand: "Amazfit", price: 7999, strap: "silicone", color: "black", category: "smart" },
|   { brand: "Samsung", price: 21999, strap: "silicone", color: "silver", category: "smart" }])
|
| {
|   acknowledged: true,
|   insertedIds: {
|     '0': ObjectId('6ac2824c64d65868db9ae85b'),
|     '1': ObjectId('6ac2824c64d65868db9ae85c'),
|     '2': ObjectId('6ac2824c64d65868db9ae85d'),
|     '3': ObjectId('6ac2824c64d65868db9ae85e'),
|     '4': ObjectId('6ac2824c64d65868db9ae85f')
|   }
| }
watchStore> |
```

Figure 21: Five smart watches inserted

Step 3: Update data

Goal: give a discount value of 1000 to every men's watch priced above 5000.

COMMAND

```
db.men.updateMany({ price: { $gt: 5000 } }, { $set: { discount: 1000 } })
```

```
watchStore> db.men.updateMany({price : { $gt:5000 } }, { $set: {discount:1000}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
watchStore> db.men.find()
[
```

Figure 22: updateMany with a price filter

Result: matchedCount: 1 and modifiedCount: 1. Only the Fossil watch (9995) qualified.

VERIFY

```
db.men.find()
```

```
watchStore> db.men.find()
[
  {
    _id: ObjectId('6ac2805b64d65068db9ae851'),
    brand: 'Titan',
    price: 4995,
    strap: 'leather',
    color: 'brown',
    category: 'men'
  },
  {
    _id: ObjectId('6ac2805b64d65068db9ae852'),
    brand: 'Fastrack',
    price: 1995,
    strap: 'silicone',
    color: 'black',
    category: 'men'
  },
  {
    _id: ObjectId('6ac2805b64d65068db9ae853'),
    brand: 'Casio',
    price: 3295,
    strap: 'steel',
    color: 'silver',
    category: 'men'
  },
  {
    _id: ObjectId('6ac2805b64d65068db9ae854'),
    brand: 'Fossil',
    price: 9995,
    strap: 'leather',
    color: 'black',
    category: 'men',
    discount: 1000
  },
  {
    _id: ObjectId('6ac2805b64d65068db9ae855'),
    brand: 'Noise',
    price: 2499,
    strap: 'silicone',
    color: 'grey',
    category: 'men'
  }
]
```

Figure 23: Fossil now has discount: 1000, other watches are unchanged

Step 4: Search data

COMMAND

```
db.men.find({ price: { $lt: 5000 } })
```

```
category: 'men'
}
watchStore> db.men.find({price : { $lt:5000}})
[
  {
    _id: ObjectId('6ac2885b64d65068db9ae851'),
    brand: 'Titan',
    price: 4995,
    strap: 'leather',
    color: 'brown',
    category: 'men'
  },
  {
    _id: ObjectId('6ac2885b64d65068db9ae852'),
    brand: 'Fastrack',
    price: 1995,
    strap: 'silicone',
    color: 'black',
    category: 'men'
  },
  {
    _id: ObjectId('6ac2885b64d65068db9ae853'),
    brand: 'Casio',
    price: 3295,
    strap: 'steel',
    color: 'silver',
    category: 'men'
  },
  {
    _id: ObjectId('6ac2885b64d65068db9ae855'),
    brand: 'Noise',
    price: 2499,
    strap: 'silicone',
    color: 'grey',
    category: 'men'
  }
]
```

Figure 24: Searching for watches priced below 5000

Result: Titan, Fastrack, Casio and Noise are returned: the budget-friendly watches in the store.

Summary of Operations

Operation	Command	Purpose
Create	db.createCollection("name")	Make a new collection
Insert	insertOne / insertMany	Add one or many documents
Read	find() / find({ filter })	Fetch all or matching documents
Update	updateOne / updateMany with \$set	Change fields in matching documents
Delete	deleteOne / deleteMany	Remove matching documents
Operators	\$gt \$lt \$in \$and \$or \$exists	Build advanced filters
Arrays	\$push	Add values to an array field

Key learnings

- A database appears in MongoDB only after it holds a collection or data.
- Use the "One" version of a command for a single document and the "Many" version for all matches.
- An update needs an operator such as \$set or \$push; the filter alone changes nothing.

- Always check `matchedCount`, `modifiedCount` and `deletedCount` to confirm what actually changed.
- Names are case-sensitive, so keep database, collection and field names consistent.
- Run `find()` with the same filter before an update or delete to preview what will be affected.

Conclusion

This project covered the complete CRUD cycle in MongoDB, from creating a database to filtering, updating and deleting data, and then applied the same skills to a realistic Watch Store. These operations are the foundation for building the backend of a MERN stack application with Express and Mongoose.